

EXPERIMENT 9

Student Name: ARNAV SHARMA

Branch: AIT-CSE-AIML

Semester: 5

Subject Name: ADBMS

UID: 24BAI70781

Section/Group: 24AIT_KRG1_B

Subject Code: 24CSP-310

Question 1

Question:

An organization stores employee details in the employee_info table. Create a PostgreSQL stored procedure that accepts an employee's **gender** as an input parameter and calculates the total number of employees belonging to that gender. The calculated count should be returned through an **OUT parameter** and displayed using RAISE NOTICE.

Solution:

```
CREATE OR REPLACE PROCEDURE sp_proc(  
    IN p_gender VARCHAR,  
    OUT p_tot_cnt INT  
)  
LANGUAGE plpgsql  
AS $$  
BEGIN  
    SELECT COUNT(*)  
    INTO p_tot_cnt  
    FROM employee_info  
    WHERE gender = p_gender;  
  
    RAISE NOTICE 'Total count of % is %',  
        p_gender, p_tot_cnt;  
END;
```

EXPERIMENT 9

\$\$;

Calling the procedure:

```
CALL sp_proc('Male', NULL);
```

The procedure counts all employees whose gender is **Male**, returns the count through p_tot_cnt, and displays it using RAISE NOTICE.

Data Output	Messages	Notifications
≡+ 📄 ▼ 📋 ▼ 🗑️ 🗄️ ⬇️ 📈 SQL		
	p_tot_cnt integer	🔒
1	6	

Question 2

Question:

An e-commerce company maintains product stock information in the products table and completed orders in the sales table. Create a PostgreSQL stored procedure that accepts a **product name** and **required quantity** as input parameters and processes the purchase only when sufficient stock is available. The procedure should calculate the sale amount as:

Sale Price = Product Price × Quantity Ordered

If sufficient stock is unavailable, the procedure should not modify any data and should display an appropriate notification.

Solution:

```
CREATE OR REPLACE PROCEDURE buy_products(
    IN p_product_name VARCHAR,
    IN p_quantity INT
)
LANGUAGE plpgsql
```

EXPERIMENT 9

AS \$\$

DECLARE

EXPERIMENT 9

```
v_product_code VARCHAR(20);  
v_price FLOAT;  
v_count INT;  
BEGIN  
  
SELECT product_code, price, quantity_remaining  
INTO v_product_code, v_price, v_count  
FROM products  
WHERE product_name = p_product_name;  
  
IF v_count >= p_quantity THEN  
  
    INSERT INTO sales  
    (order_date, product_code, quantity_ordered, sale_price)  
    VALUES  
    (CURRENT_DATE,  
    v_product_code,  
    p_quantity,  
    v_price * p_quantity);  
  
    UPDATE products  
    SET quantity_remaining = quantity_remaining - p_quantity,  
        quantity_sold = quantity_sold + p_quantity  
    WHERE product_code = v_product_code;  
  
    RAISE NOTICE
```

EXPERIMENT 9

'PRODUCT SOLD. Order placed successfully for % unit(s) of %.',

p_quantity, p_product_name;

ELSE

RAISE NOTICE

'INSUFFICIENT QUANTITY. Order cannot be processed for % unit(s) of %.',

p_quantity, p_product_name;

END IF;

END;

\$\$;

Calling the procedure:

CALL buy_products('Samsung Galaxy S23 Ultra', 3);

Data Output Messages Notifications

NOTICE: PRODUCT SOLD..! Order p Notifications essfully for 3 unit(s) of Samsung Galaxy S23 Ultra.
CALL

Query returned successfully in 82 msec.

✓ Query returned successfully in 82 msec. ✕